Time and Space Complexity

1. What is time complexity?

The time required by the algorithm to solve a given problem is called time complexity of the algorithm.

Or

The total number of iterations required to solve a given problem is called time complexity of the algorithm.

2. Why do we need time complexity?

1. *It helps in comparing different algorithms to determine which one is more efficient for a specific task.*
2. *It predicts how it will perform as the input size increases.It is valuable for estimating the resources(time and space) required for the algorithm to handle input.*

3. Types of Algorithm analysis?

1. *Worst case*
   The maximum time an algorithm takes to execute for any input.
2. *Best case*
   The minimum time an algorithm takes to execute for any input.
3. *Average case*
   The Average time an algorithm takes to execute for any input.

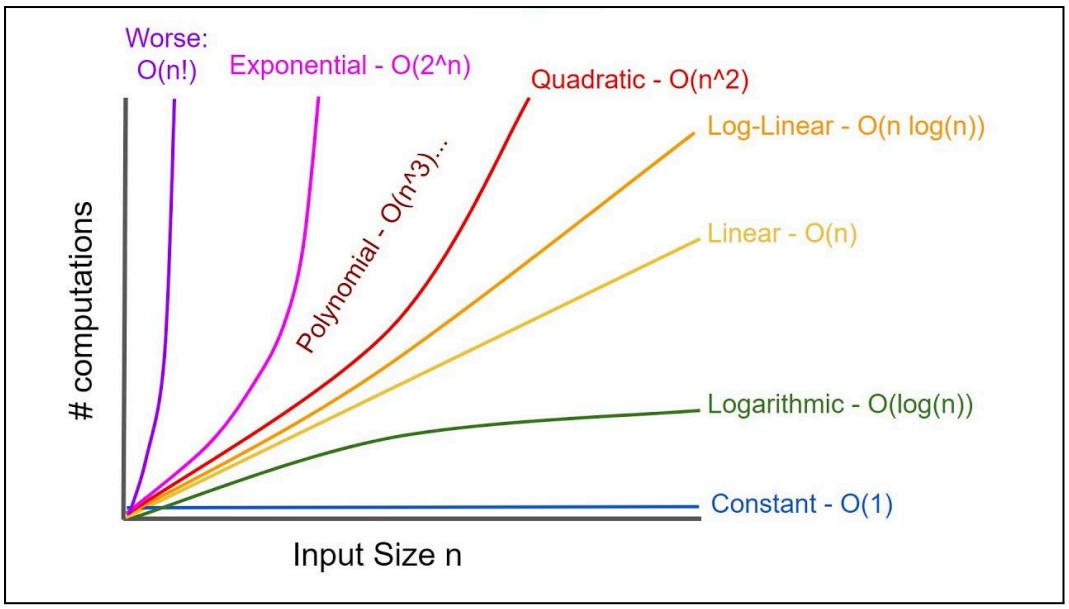
4. What are asymptotic Notations?

Asymptotic notations are the mathematical tools to determine the time complexity of an algorithm.

There are mainly three asymptotic notations:

1. *Big-O Notation (It represent worst case)*
2. *Omega Notation (It represent best case)*
3. *Theta Notation (It represent average case)*

5. Big O Notation graph?



$O(1) > O(logn) > O(n) > O(nlogn) > O(n^2) > O(2^n) > O(n!)$

5. Some common time complexities:

O(1):

```
for (int i = 0; i < 5; i++){
  cout<<"Hello World";
}
//This loop will run only 5 times
// Total T.C = O(1)
```

O(logn):

```
for (int i = 0; i < n; i = i*2){
  count++;
}
// Total T.C = O(log(n))
```

O(n):

```
for (int i = 0; i < n; i++){  //This loop will run n times
   cout<<"Hello world";
}
// Total T.C = O(n)
```

O(n):

```
for (int i = 0; i < n; i++){  //This loop will run n times
    cout<<"Hello world";
}

for (int i = 0; i < n; i++){  //This loop will run n times
    cout<<"Good Morning";
}
// Total T.C = O(n + n) => O(2n) => O(n)
```

O(n+m):

```
for (int i = 0; i < n; i++){  //This loop will run n times
    cout<<"Hello world";
}

for (int i = 0; i < m; i++){  //This loop will run m times
    cout<<"Good Morning";
}
// Total T.C = O(n + m)
```

O(nlogn):

```
for (int i = 0; i < n; i=i*2) //This loop will run logn times
{
    for(int i = 0; i < n; i = i++){ //This loop will run n times
        count++;
    }
}
// Total T.C = O(nlogn)
```

O(n^2):

```
for (int i = 0; i < n; i++){  //This loop will run n times
    for (int j = 0; j < n; j++){  //This loop will run n times
        count++;
    }
}
// Total T.C = O(n*n) => O(n^2)
```

$O(n^2)$:

```
for (int i = 0; i < n; i++){  //This loop will run n times
    for (int j = 0; j < i; j++){  //This loop will run i times
        count++;
    }
}
// Total T.C = O(n*n) => O(n^2)
```

$O(n*m)$:

```
for (int i = 0; i < n; i++){  //This loop will run n times
    for (int j = 0; j < m; j++){  //This loop will run m times
        count++;
    }
}
// Total T.C = O(n*m)
```

$O(n*m)$:

```
for (int i = 0; i < n; i++){  //This loop will run n times
    for (int j = i; j < m; j++){  //This loop will run m times
        count++;
    }
}
// Total T.C = O(n*m)
```

$O(n^3)$:

```
for (int i = 0; i < n; i++){          //This loop will run n times
    for (int j = i; j < n; j++){      //This loop will run n times
        for (int k = 0; k < n; k++){  //This loop will run n times
            count++;
        }
    }
}
// Total T.C = O(n*n*n) => O(n^3)
```

$O(n*m*p)$:

```
for (int i = 0; i < n; i++){          //This loop will run n times
    for (int j = i; j < m; j++){      //This loop will run m times
        for (int k = 0; k < p; k++){  //This loop will run p times
            count++;
        }
    }
}
// Total T.C = O(n*m*p)
```

6. What is Space complexity?

The space Complexity of an algorithm is the total space taken by the algorithm with respect to the input size. Space complexity includes both Auxiliary space and space used by input.

7. Some common time complexities:

O(1):

```
int a, b;
cin>>a>>b;
cout<<a + b;
//S.C = O(1)
```

O(1):

```
vector<int> v(10);
for (int i = 0; i < v.size(); i++){
    v.push_back(i);
}
//S.C = O(1)
```

O(n):

```
vector<int> v;
for (int i = 0; i < v.size(); i++){
    v.push_back(i);
}
//S.C = O(n)
```

O(n^2):

```
int n,m;
int arr[n][m];
for (int i = 0; i < m; i++){
    for (int j = 0; j < n; j++){
        arr[i][j] = i + j;
    }
}
//S.C = O(n^2)
```